

Cours 4

Programmation orientée objet

Jérémy Kairis

Les collections

Les collections

Pourquoi ?

Les tableaux ont un inconvénient majeur : leur taille est fixe, ce qui signifie qu'ils ne peuvent pas contenir plus d'éléments que ce qui a été défini à leur création.

Cependant, dans de nombreuses situations, notamment lorsque l'on importe des données externes telles que des bases de données ou des services web, il est impossible de prévoir le nombre d'éléments à l'avance.

C'est pourquoi nous avons besoin de structures de données flexibles capables de s'ajuster dynamiquement en fonction des besoins.

Les collections

ArrayList

L'***ArrayList*** est une option lorsque vous avez besoin de stocker des données de manière flexible et que vous êtes prêt à gérer les conversions de type associées.

Cependant, si vous connaissez le type d'objets que vous allez stocker, il est recommandé d'utiliser des collections fortement typées telles que **List<T>**.

```
var maListe = new ArrayList();

// Ajout d'éléments de différents types
maListe.Add(42);
maListe.Add("Bonjour");
maListe.Add(3.14);

// Accès aux éléments
Console.WriteLine("Éléments de l'ArrayList :");
foreach (var element in maListe)
{
    Console.WriteLine(element);
}
```

Les collections

HashTable

Une “***Hashtable***” est comparable à un dictionnaire où l’on accède à une valeur en utilisant une clé. La clé et la valeur sont de type “***object***”. De plus, la clé doit être unique.

Chaque élément de la “***Hashtable***” est de type “***DictionaryEntry***” proposant les propriétés ***Key*** et ***Value*** toutes deux de type ***object***.

Cependant, si vous connaissez le type d'objets que vous allez stocker, il est recommandé d'utiliser le **Dictionary<TKey, TValue>**.


```
var maHashtable = new Hashtable();
```

```
maHashtable["cle1"] = 42;
```

```
maHashtable["cle2"] = "Bonjour";
```

```
maHashtable["cle3"] = 3.14;
```

```
Console.WriteLine("Valeur associée à la clé 'cle1' : " + maHashtable["cle1"]);
```

```
foreach (DictionaryEntry entry in maHashtable)
```

```
{
```

```
    Console.WriteLine("Clé : " + entry.Key + ", Valeur : " + entry.Value);
```

```
}
```

Les collections

Génériques

Les collections ont un avantage sur les tableaux, elles sont de taille variable.

Mais en les utilisant, nous avons perdu le typage fort des tableaux.

Les collections génériques, elles ont les avantages d'être de taille dynamique et d'être également typée.

Les collections

List<T>

La “***List<T>***” remplace l’“***ArrayList***”.

Le type est spécifié lors de la déclaration.

Une fois ce type spécifié, les éléments insérer et récupérer seront de ce type.

```
var maListe = new List<string>
{
    "Valeur 1",
    "Valeur 2",
    "Valeur 3"
};
```

```
Console.WriteLine("Élément à l'indice 0 : " + maListe[0]);
```

```
Console.WriteLine("\nContenu de la liste :");
foreach (var element in maListe)
{
    Console.WriteLine("Valeur : " + element);
}
```

Les collections

Dictionary<T, U>

Un “*Dictionary*<T, U>” est comparable à une “*HashTable*”. Cependant les types de la clé et de la valeur seront spécifiés.

Chaque élément sera donc de type “*KeyValuePair*<T, U>”.


```
var monDictionnaire = new Dictionary<string, int>
{
    ["cle1"] = 42,
    ["cle2"] = 123,
    ["cle3"] = 99
};
```

```
Console.WriteLine("Valeur associée à la clé 'cle1' : " + monDictionnaire["cle1"]);
```

```
Console.WriteLine("\nContenu du Dictionary :");
foreach (KeyValuePair<string, int> paire in monDictionnaire)
{
    Console.WriteLine("Clé : " + paire.Key + ", Valeur : " + paire.Value);
}
```

Les collections

Exercices

1. Grâce à une boucle “***while***” et à l’aide d’une collection, calculez les nombres premiers inférieur à un nombre entier entré au clavier.
2. Grâce à une boucle “***for***” et à l’aide d’une collection générique, calculez les x premiers nombres premiers.

```
var x = int.Parse(Console.ReadLine());

DisplayPrimeNumbersBefore(x);
DisplayXPrimeNumbers(x);

void DisplayPrimeNumbersBefore(int x)
{
    var number = x;
    List<int> primeNumbers = new();
    while (number-- > 0)
        if (IsPrime(number))
            primeNumbers.Add(number);

    ShowPrimeNumbers($"Voici les nombres premiers avant {x}", primeNumbers);
}

void DisplayXPrimeNumbers(int x)
{
    List<int> primeNumbers = new();
    for (int i = 0; primeNumbers.Count < x; i++)
        if (IsPrime(i))
            primeNumbers.Add(i);

    ShowPrimeNumbers($"Voici les {x} premiers nombres premiers", primeNumbers);
}

bool IsPrime(int n)
{
    for (var i = 2; i < n; i++)
        if (n % i == 0)
            return false;
    return n > 1;
}

void ShowPrimeNumbers(string message, List<int> primeNumbers)
{
    Console.WriteLine(message);
    foreach (var numbers in primeNumbers)
        Console.WriteLine(numbers);
}
```


Les structures

Les structures

struct

Une structure permet de déclarer un type complexe de type **valeur**. La déclaration d'une structure se fait à l'aide du mot-clé “***struct***”.

Elles fonctionnent comme les types valeurs. Les structures sont copiées lors de l'assignation.

Lorsqu'une structure est assigné à une nouvelle variable, toutes les données sont copiées et les modifications apportées à la nouvelle copie ne changent pas les données de la structure d'origine.


```
Point point1;  
point1.X = 10;  
point1.Y = 20;
```

```
Point point2 = point1;  
point2.X = 30;
```

```
Console.WriteLine("Point1 - X: " + point1.X + ", Y: " + point1.Y);  
Console.WriteLine("Point2 - X: " + point2.X + ", Y: " + point2.Y);
```

```
struct Point  
{  
    public int X;  
    public int Y;  
}
```

Les structures

Exercice

Définissez deux structures Celsius et Fahrenheit toutes deux ayant une variable température de type “***double***”.

Ajoutez une méthode de conversion en Celsius dans la structure Fahrenheit et inversement.

```
Celcius cel;  
cel.temperature = 12;  
Console.WriteLine(cel.ToFahrenheit().temperature);
```

```
Fahrenheit far;  
far.temperature = 53.6;  
Console.WriteLine(far.ToCelcius().temperature);
```

```
struct Celcius  
{  
    public double temperature;  
    public Fahrenheit ToFahrenheit()  
    {  
        Fahrenheit fahrenheit;  
        fahrenheit.temperature = temperature * 9 / 5 + 32;  
        return fahrenheit;  
    }  
}
```

```
struct Fahrenheit  
{  
    public double temperature;  
    public Celcius ToCelcius()  
    {  
        Celcius celcius;  
        celcius.temperature = (temperature - 32) * 5 / 9;  
        return celcius;  
    }  
}
```


Les énumérations

Les énumérations

Déclaration

Les énumérations nous permettent de stocker de façon simple plusieurs valeurs invariables qui sont liées logiquement les unes aux autres.

Les énumérations se déclare avec le mot clé “***enum***” et les valeurs ont une valeur numérique par défaut commençant à 0 et qui peut être redéfinie.

```
enum Carburant
```

```
{
```

```
    Essence,
```

```
    Diesel,
```

```
    Gaz,
```

```
    Electricité
```

```
}
```


Les énumérations

Utilisation

Une fois la valeur de la variable définie, les valeur numériques et littérales sont accessible.

“***GetNames***” retourne la liste des valeurs du type de l’énumération passée en paramètre sous forme de “***string[]***”.

“***Parse***” & “***TryParse***” permettent de convertir une “***string***” en une valeur de type de l’énumération.

```
string[] carburants = Enum.GetNames(typeof(Carburant));  
foreach (string carburant in carburants)  
{  
    Console.WriteLine(carburant);  
}  
  
var car = "Diesel";  
var carEnum = (Carburant)Enum.Parse(typeof(Carburant), car);  
  
Console.WriteLine("Carburant : " + carEnum);  
Console.WriteLine("Carburant entier : " + (Carburant)2);  
  
enum Carburant { Essence, Diesel, Gaz, Electricité }
```


Les énumérations

Exercices

Décrivez une structure permettant de construire une voiture en utilisant des valeurs présentes dans des énumérations.

Ensuite, ajoutez lui des variables et méthodes utiles.

Une fois construite, faites la rouler.

Les classes

Les classes

Introduction

Une classe est essentiellement un modèle à partir duquel il est possible de créer des objets de type référence.

Chacune pourra contenir des variables, des propriétés, des indexeurs, des méthodes, des constructeurs, un destructeur, des délégués, des événements, voir même des structures et/ou d'autres classes.

Une classe à pour but de définir quelles sont les données et fonctionnalités que chaque objet pourra contenir.

Pour définir une classe, nous utiliserons le mot clé « **class** ».

Les classes

new

Toute variable de type référence doit être instanciée. C'est la création d'une instance de la classe en mémoire.

Pour créer un nouvel objet en mémoire, c'est le mot-clé « ***new*** ».

Ce mot-clé appelle le constructeur de l'objet auquel on peut passer les arguments nécessaires à sa construction.

// Création de 2 instances de la classe Personne

```
Personne personne1 = new();  
var personne2 = new Personne();
```

// Définition de la classe Personne

```
class Personne
```

```
{
```

```
}
```

```
Personne p = new("Kairis", "Jérémy");
```

```
Console.WriteLine(p.Nom);
```

```
Console.WriteLine(p.Prenom);
```

```
// Définition de la classe Personne avec son constructeur primaire
```

```
class Personne(string nom, string prenom)
```

```
{
```

```
    public string Nom = nom;
```

```
    public string Prenom = prenom;
```

```
}
```


Les classes

partial

Une même classe peut être déclarée sur plusieurs fichiers, en utilisant le mot-clé « *partial* ».

C'est à la compilation que le compilateur se charge alors de rassembler les fichiers en une classe unique.

```
partial class Personne
{
    string nom;
}
```

```
partial class Personne
{
    int age;
}
```


Les classes statiques

static

Si nous utilisons le modificateur « ***static*** » pour déclarer un membre d'une classe ou d'une structure, cela revient à dire qu'il appartient au type lui-même plutôt qu'à une instantiation spécifique.

Nous accédons aux membres statiques en utilisant le nom du type.

Le mot-clé « ***static*** » peut être utilisé avec des classes, des champs, des méthodes, des propriétés, des événements et des constructeurs.

```
Exemple.Compteur++;
```

```
static class Exemple  
{  
    public static int Compteur = 0;  
}
```

Les classes statiques

Utilisation

Une classe statique est fondamentalement identique à une classe non statique, à la différence près qu'une classe statique ne peut pas être instanciée.

En d'autres termes, nous ne pouvons pas utiliser le mot clé « **~~new~~** » pour créer une variable du type classe.

Une classe statique ne contient uniquement que des membres statiques. Par conséquent, comme il n'y a aucune variable d'instance, nous accédons aux membres d'une classe statique en utilisant le nom de classe lui-même.

Les classes statiques

Exemple

Une classe statique peut être utilisée comme conteneur commode pour les ensembles de méthodes qui opèrent simplement sur des paramètres d'entrée et n'ont pas à obtenir ou définir de champs d'instance internes comme :

- La classe « ***Math*** » ou « ***Console*** ».
- Une classe pour la centralisation des variables de configuration

```
Math.Sqrt(16);
```

```
Math.Abs(-42);
```

```
Math.Round(3.75);
```

```
Math.Pow(2.0, 3);
```

```
Console.WriteLine("");
```

```
Console.ReadLine();
```

Les classes

Exercices

- Transformer la structure voiture en classe, l'instancier et la faire rouler.
- Définir et utiliser les classes imaginées au premier cours.